

HarvardX Capstone My Own Project - US Census Data

Frank Valdivia

2025-08-30

Contents

1	INTRODUCTION	2
2	ANALYSIS AND METHODS	4
2.1	Loading libraries	5
2.2	Unzipping donloaded file and Generating datasets	6
2.3	Analysis and methods	11
2.3.1	Exploratory Analysis	12
2.3.2	Accuracy and the Confusion Matrix	15
2.3.3	Models	17
2.3.3.1	First model: KNN	18
2.3.3.2	Second model: KNN with Cross Validation	22
2.3.3.3	Third model: Decision Tree	27
2.3.3.4	Fourth model: Decision Tree with Complexity Parameter	31
2.3.3.5	Fifth model: Random Forest	36
2.3.3.6	Sixth model: Random Forest with Cross Validation	42
3	RESULTS	46
4	CONCLUSION	50
5	REFERENCES	51

1 INTRODUCTION

The goal of this project is to build a CLASSIFICATION model that will predict INCOME LEVEL for the 1994 US Census Data (1) downloaded from The UCI Machine Learning Repository.

Six classification models will be built and each model assessed against its Accuracy. The goal is to achieve more than 80% Accuracy and the model with the highest Accuracy on the train set will be selected.

The US Census dataset has 14 columns of which “Income Level” is the outcome. Income has two levels: “<=50K” “>50K”

According to the UCI website (<https://archive.ics.uci.edu/dataset/2/adult>):

“Extraction was done by Barry Becker from the 1994 Census database. A set of reasonably clean records was extracted using the following conditions: ((AAGE>16) && (AGI>100) && (AFNLWGT>1)&& (HR-SWK>0))”

This dataset has 14 columns of which “Income” is the outcome. Income has two levels:

“<=50K” “>50K”

The following is the list of the other features that the dataset has:

- age: continuous.
- workclass: Private, Self-emp-not-inc, Self-emp-inc, Federal-gov, Local-gov, State-gov, Without-pay, Never-worked.
- fnlwgt: continuous.
- education: Bachelors, Some-college, 11th, HS-grad, Prof-school, Assoc-acdm, Assoc-voc, 9th, 7th-8th, 12th, Masters, 1st-4th, 10th, Doctorate, 5th-6th, Preschool.
- education-num: continuous.
- marital-status: Married-civ-spouse, Divorced, Never-married, Separated, Widowed, Married-spouse-absent, Married-AF-spouse.
- occupation: Tech-support, Craft-repair, Other-service, Sales, Exec-managerial, Prof-specialty, Handlers-cleaners, Machine-op-inspct, Adm-clerical, Farming-fishing, Transport-moving, Priv-house-serv, Protective-serv, Armed-Forces.
- relationship: Wife, Own-child, Husband, Not-in-family, Other-relative, Unmarried.
- race: White, Asian-Pac-Islander, Amer-Indian-Eskimo, Other, Black.
- sex: Female, Male.
- capital-gain: continuous.
- capital-loss: continuous.
- hours-per-week: continuous.
- native-country: United-States, Cambodia, England, Puerto-Rico, Canada, Germany, Outlying-US(Guam-USVI-etc), India, Japan, Greece, South, China, Cuba, Iran, Honduras, Philippines, Italy, Poland, Jamaica, Vietnam, Mexico, Portugal, Ireland, France, Dominican-Republic, Laos, Ecuador, Taiwan, Haiti, Columbia, Hungary, Guatemala, Nicaragua, Scotland, Thailand, Yugoslavia, El-Salvador, Trinidad&Tobago, Peru, Hong, Holand-Netherlands.

A subset of features will be used to build the model

+++++++ IMPORTANT +++++++

To run this report, the file adult.zip from the UCI website should be in your working directory.

The adult.zip file was copied from the UCI website (UC Irvine Machine Learning Repository) to a GitHub folder.

This file will be downloaded to your working folder.

It may happen that the download does not work. If that is the case, please go to the following GitHub page and download adult.zip manually and copy it to your working directory.

<https://github.com/frankvaldivia/Capstone-FV-DataScience/tree/main>

Or Download it from the UCI website

<https://archive.ics.uci.edu/dataset/2/adult>

Because processing time to generate the models was significant (more than one day), the six models evaluated were generated and saved as RDS files in GitHub. The RDS files are downloaded into your working directory:

train_knn.rds: KNN model

train_knn_cv.rds: KNN model with Cross Validation Model

train_dt.rds: Decision Tree model

train_dt_cp.rds: Decision Tree - Complexity Parameter model

train_rf.rds: Random Forest model

train_rf_2.rds: Random Forest with Cross Validation model

The GitHub folder with the adult.zip and RDS files (models) is as follows:

<https://github.com/frankvaldivia/Capstone-FV-DataScience/tree/main>

+++++

TO RUN THIS REPORT THE MODELS ALREADY GENERATED AND DOWNLOADED AS RDS FILES WILL BE USED. THESE RDS FILES WERE GENERATED USING THE R CODE FOR THIS PROJECT.

Machine learning methods will be used, which means that two sub datasets will be created during this process:

A train set will be generated with the name: censusTrain; this dataset will be used to train the model.

A test set will be generated with the name: censusTest; this dataset will be used to test the model trained using the train dataset.

The output will be a classification model and Accuracy will be used to measure quality of the model.

2 ANALYSIS AND METHODS

The overall process is as follows:

- 2.1. Loading libraries
- 2.2. Unzipping donloaded file and Generating datasets
- 2.3. Analysis and methods

2.1 Loading libraries

To run the report the following R libraries need to be previously installed:

- `library(tidyverse)`
- `library(caret)`
- `library(ggplot2)`
- `library(dplyr)`
- `library(lattice)`
- `library(gridExtra)`
- `library(beepr)`
- `library(dslabs)`
- `library(rpart)`
- `library(rpart.plot)`
- `library(randomForest)`
- `library(Rborist)`
- `library(class)`
- `library(grid)`

Above libraries are loaded.

2.2 Unzipping donloaded file and Generating datasets

+++++++ IMPORTANT +++++++

Because processing time to generate the models was significant (more than one day), the six models evaluated were generated and saved as RDS files in GitHub. These files (models) were downloaded into your working directory:

train_knn.rds: KNN model
train_knn_cv.rds: KNN model with Cross Validation Model
train_dt.rds: Decision Tree model
train_dt_cp.rds: Decision Tree - Complexity Parameter model
train_rf.rds: Random Forest model
train_rf_2.rds: Random Forest wth Cross Validation model

GitHub folder:

<https://github.com/frankvaldivia/Capstone-FV-DataScience/tree/main>

+++++++

adult.zip has two files, The author of adult.zip already generated a train dataset and a test dataset. In this project, both files will be put together to create a new whole Census_data dataset, which will be used to create a new train dataset and a new test dataset.

Following,

1. Column names are assigned.
2. Empty rows are removed.
3. Temporary train and test data sets are created.

```
beep()

# assigning column names to both temporary files

colnames(census_train) <- c("age", "workclass", "weight", "education",
                           "educationlevel", "maritalstatus", "occupation",
                           "relationship", "race", "sex", "capitalgain",
                           "capitalloss", "hoursperweek", "nativecountry",
                           "income")

colnames(census_test) <- c("age", "workclass", "weight", "education",
                           "educationlevel", "maritalstatus", "occupation",
                           "relationship", "race", "sex", "capitalgain",
                           "capitalloss", "hoursperweek", "nativecountry",
                           "income")

# removing empty rows

temp_train<-census_train[-c(32562,32563),]

temp_test<-census_test[-c(1,16283,16284),]
```

Temporary train and test data sets are created with the following structure and number of rows. These two data sets were originally created by the author and will be used to create a new master census data for this project.

```
##  
## Temporary train set: Columns
```

x
age
workclass
weight
education
educationlevel
maritalstatus
occupation
relationship
race
sex
capitalgain
capitalloss
hoursperweek
nativecountry
income

```
##  
## Temporary train set: Number of Records
```

dim(temp_train)
32561

```
##  
## Temporary test set: Columns
```

x
age
workclass
weight
education
educationlevel
maritalstatus
occupation
relationship
race
sex
capitalgain
capitalloss
hoursperweek
nativecountry
income

```
##  
## Temporary test set: Number of Records
```

dim(temp_test)

16281

```
##
## New Census data: Columns
```

x

age
workclass
weight
education
educationlevel
maritalstatus
occupation
relationship
race
sex
capitalgain
capitalloss
hoursperweek
nativecountry
income

```
##
## New Census data: Number of Records
```

dim(census_data)

48842

With the whole new Census Data set, predictors (columns) are redefined as Factors or Integers. The outcome (Y) was redefined as a string with two values: “<=50K” “>50K”

Cleaning Census data continued with the removal of columns that will not be used in the analysis, empty records and the values for the outcome.

The census data set and columns to be used in this project are as follows:

```
##
## Clean Census data: Columns
```

x

age
workclass
education
maritalstatus
occupation
relationship
race
sex
hoursperweek
income

```
##
```

```
## clean Census data: Head
```

age	workclass	education	maritalstatus	occupation	relationship	race	sex	hoursperweek	income
24	State-gov	Bachelors	Never-married	Adm-clerical	Not-in-family	White	Male	36	<=50K
35	Self-emp-not-inc	Bachelors	Married-civ-spouse	Exec-managerial	Husband	White	Male	6	<=50K
23	Private	HS-grad	Divorced	Handlers-cleaners	Not-in-family	White	Male	36	<=50K
38	Private	11th	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	36	<=50K
13	Private	Bachelors	Married-civ-spouse	Prof-specialty	Wife	Black	Female	36	<=50K
22	Private	Masters	Married-civ-spouse	Exec-managerial	Wife	White	Female	36	<=50K

```
##
```

```
## clean Census data: Number of Records
```

```

dim(census_data)
48842

```

```
##
```

```
## clean Census data: levels for every predictor and outcome(Income)
```

```
## $age
```

```
## NULL
```

```
##
```

```
## $workclass
```

```
## [1] "?" "Federal-gov" "Local-gov" "Never-worked"
```

```
## [5] "Private" "Self-emp-inc" "Self-emp-not-inc" "State-gov"
```

```
## [9] "Without-pay"
```

```
##
```

```
## $education
```

```
## [1] "Preschool" "1st-4th" "5th-6th" "7th-8th" "9th"
```

```
## [6] "10th" "11th" "12th" "HS-grad" "Assoc-acdm"
```

```
## [11] "Assoc-voc" "Some-college" "Bachelors" "Masters" "Prof-school"
```

```
## [16] "Doctorate"
```

```
##
```

```
## $maritalstatus
```

```
## [1] "Divorced" "Married-AF-spouse" "Married-civ-spouse"
```

```
## [4] "Married-spouse-absent" "Never-married" "Separated"
```

```
## [7] "Widowed"
```

```
##
```

```
## $occupation
```

```
## [1] "?" "Adm-clerical" "Armed-Forces"
```

```
## [4] "Craft-repair" "Exec-managerial" "Farming-fishing"
```

```
## [7] "Handlers-cleaners" "Machine-op-inspct" "Other-service"
```

```
## [10] "Priv-house-serv" "Prof-specialty" "Protective-serv"
```

```
## [13] "Sales" "Tech-support" "Transport-moving"
```

```
##
```

```
## $relationship
```

```

## [1] "Husband"      "Wife"          "Unmarried"     "Not-in-family"
## [5] "Other-relative" "Own-child"
##
## $race
## [1] "Amer-Indian-Eskimo" "Asian-Pac-Islander" "Black"
## [4] "Other"              "White"
##
## $sex
## [1] "Female" "Male"
##
## $hoursperweek
## NULL
##
## $income
## [1] "<=50K" ">50K"

```

2.3 Analysis and methods

First, exploratory analysis will be carried out showing frequencies for predictors and the outcome (Income).

Then, a description of the measure of success (accuracy) will be included before elaborating on the models applied to Census data.

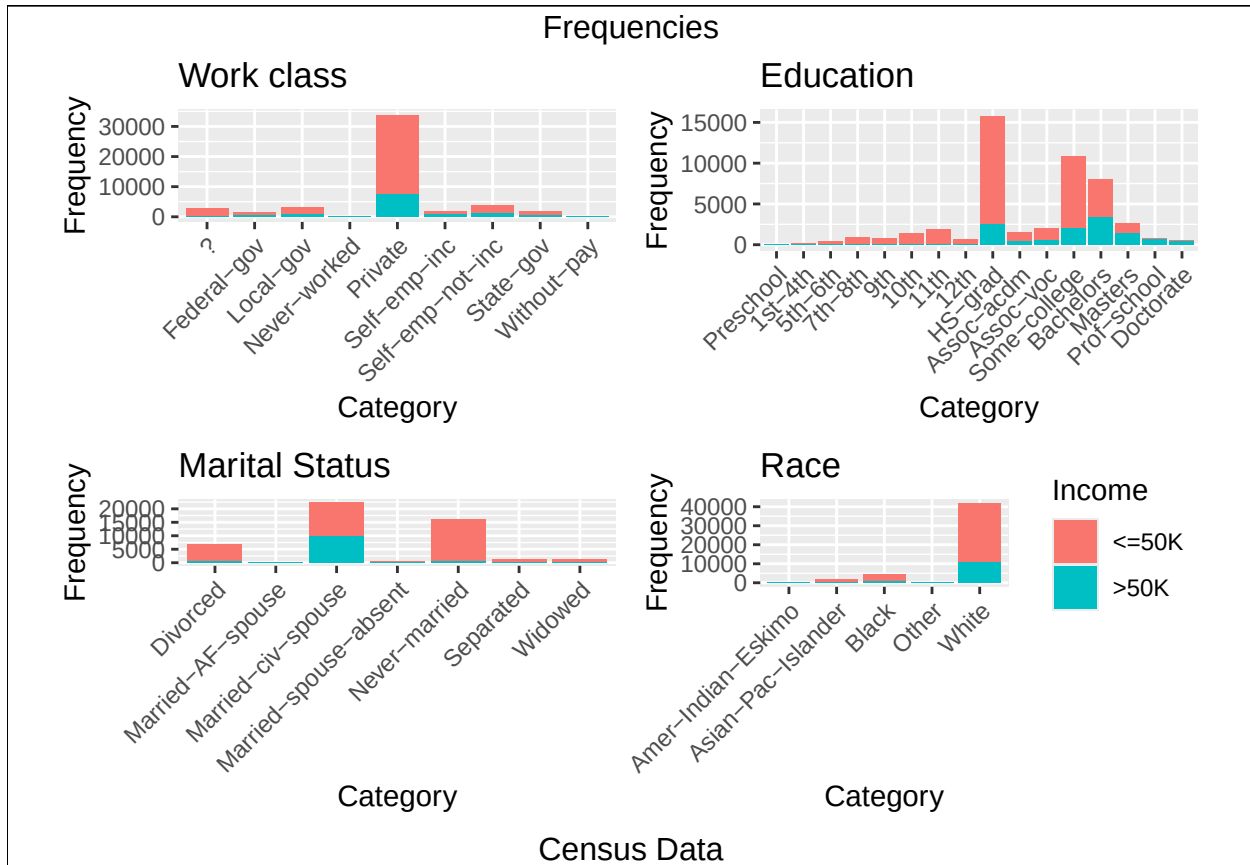
Every model will include specific results and plots and all will have their Accuracy estimated and compared.

The goal is to have a model with 80% accuracy or greater.

2.3.1 Exploratory Analysis

Following are the frequencies for the following predictors. They are all categories:

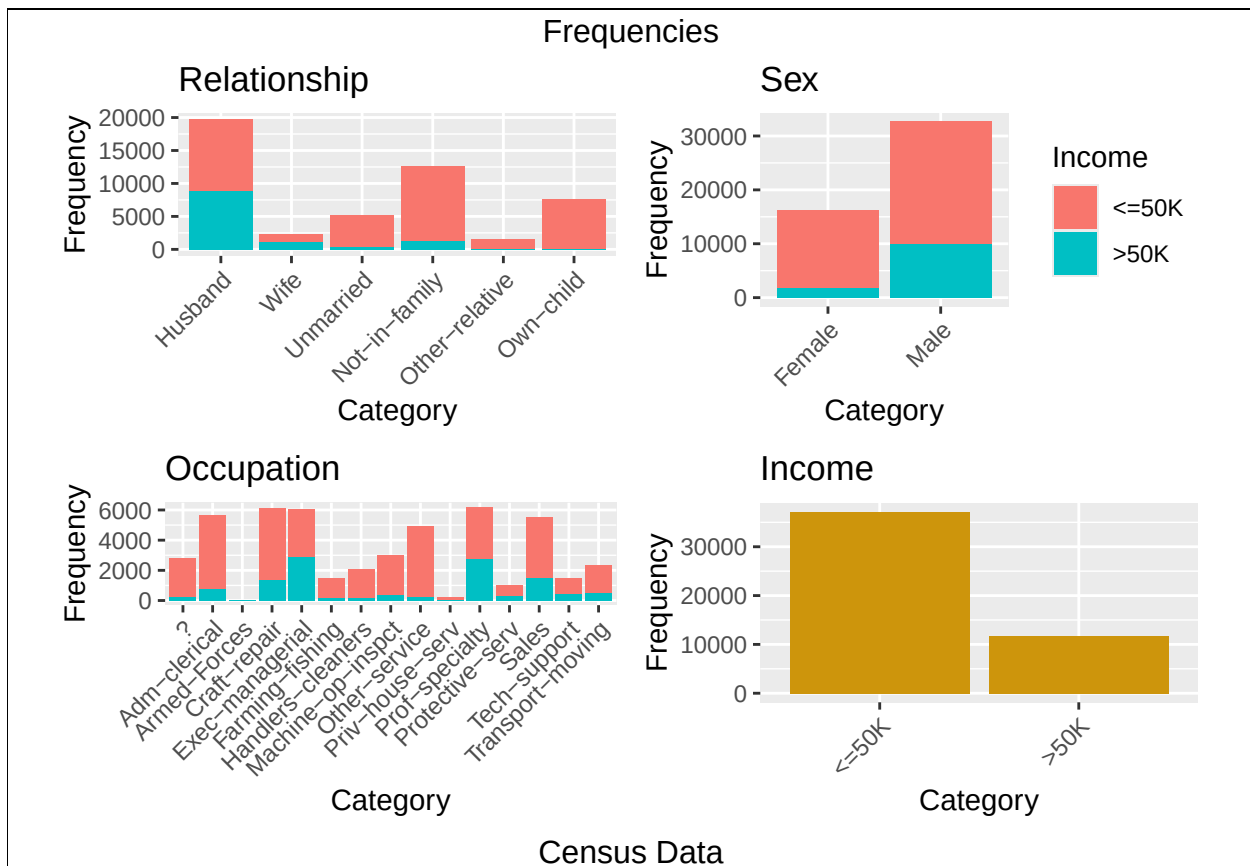
- Work class
- Education
- Marital Status
- Race



Previous plots suggest that there is some level of dependency of income on some specific categories, i.e. HS-Grad, Bachelors, Some College in Education, or Private in Work class.

Following are the frequencies for the following predictors. They are all categories:

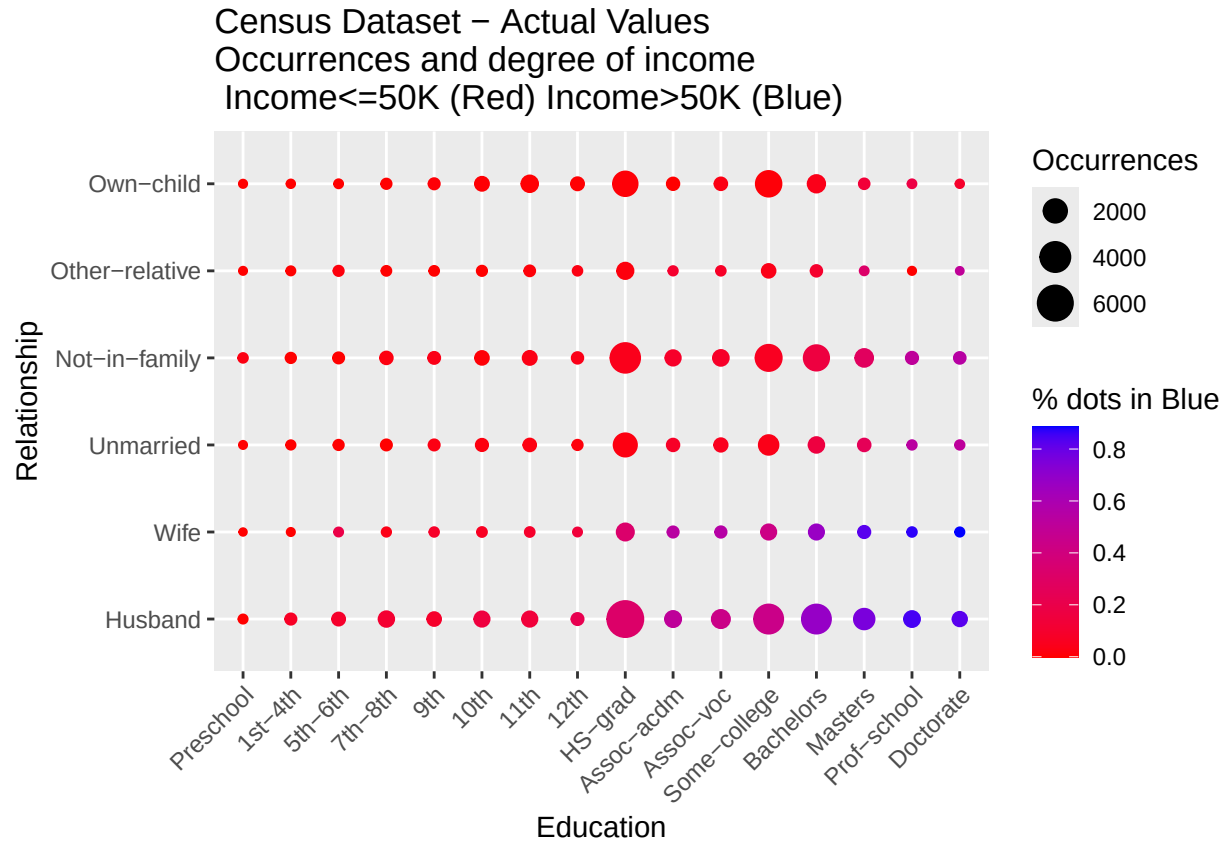
- Relationship
- Sex
- Occupation
- Income



Previous plots suggest that there is some level of dependency of income on some specific categories, i.e. Female in Sex, or Own-child in Relationship.

The following graph shows the actual values of Income for the whole Census dataset. Because this is a two dimensional graph, only two predictors out of the nine are shown, Education and Relationship.

Every dot shows the frequency of occurrences that have “>50K” as income in blue as a percentage of the total number of records for that combination of Education and Relationship. The size of the dot represents the number of records for that combination of Education and Relationship.



2.3.2 Accuracy and the Confusion Matrix

The accuracy of every model will be calculated and compared. Accuracy is the percentage of correct Income predictions on the Census data set.

$$Accuracy = \frac{RightPredictions}{TotalPredictions}$$

The confusion Matrix will also be reported for every model. The confusion Matrix from R summarizes the performance of the model. It compares the predicted values against the train set values.

To run the models and get the confusion matrix, two sets were created from the Census data set. The train set and the test set, which is 30% of the whole Census data set.

Following are the structure and number of rows that each of these datasets has:

##

censusTrain: Head

age	workclass	education	maritalstatus	occupation	relationship	race	sex	hoursperweek	income
24	State-gov	Bachelors	Never-married	Adm-clerical	Not-in-family	White	Male	36	<=50K
35	Self-emp-not-inc	Bachelors	Married-civ-spouse	Exec-managerial	Husband	White	Male	6	<=50K
23	Private	HS-grad	Divorced	Handlers-cleaners	Not-in-family	White	Male	36	<=50K
38	Private	11th	Married-civ-spouse	Handlers-cleaners	Husband	Black	Male	36	<=50K
13	Private	Bachelors	Married-civ-spouse	Prof-specialty	Wife	Black	Female	36	<=50K
22	Private	Masters	Married-civ-spouse	Exec-managerial	Wife	White	Female	36	<=50K

##

censusTrain: Number of Records

dim(censusTrain)
34190

##

censusTest: Head

	age	workclass	education	maritalstatus	occupation	relationship	race	sex	hoursperweek	income
7	34	Private	9th	Married-spouse-absent	Other-service	Not-in-family	Black	Female	9	<=50K
8	37	Self-emp-not-inc	HS-grad	Married-civ-spouse	Exec-managerial	Husband	White	Male	41	>50K
27	4	Private	HS-grad	Never-married	Craft-repair	Own-child	White	Male	36	<=50K
33	30	Private	Bachelors	Divorced	Exec-managerial	Own-child	White	Male	36	<=50K
39	16	Private	Some-college	Married-civ-spouse	Sales	Husband	White	Male	33	>50K

	age	workclass	education	maritalstatus	occupation	relationship	race	sex	hoursperweek	income
42	38	Self-emp-not-inc	Bachelors	Married-civ-spouse	Prof-specialty	Husband	White	Male	36	<=50K

##

censusTest: Number of Records

dim(censusTest)
14652

2.3.3 Models

The following steps will be followed with all six classification models:

1. Predictions using the Train set will be estimated
2. Plots will be generated
3. The confusion matrix, including Accuracy, will be produced and reported comparing the predictions on the train set to the actual values of the train set.

Six models will be built and each model assessed against its Accuracy.

The goal is to achieve more than 80% Accuracy and the model with the highest Accuracy on the train set will be selected. Accuracy using the test set will be calculated to confirm the goal of greater than 80%.

1. KNN (K-Nearest Neighbors): It classifies data points based on their similarity to existing data points. The similarity is measured through the euclidian distance and the minimum distance is chosen to predict the values. During the first running, this model was saved in a file (train_knn.rds) and uploaded to GitHub.
2. KNN with Cross Validation Model: It is a KNN model that uses tuning to find the optimum value of K. During the first running, this model was saved in a file (train_knn_cv.rds) and uploaded to GitHub.
3. Decision Tree: It splits data into subsets based on a series of conditions identified based on minimizing a coefficient. Each internal node represents a test (Yes or No) on a predictor, each branch represents the outcome of the test, and each leaf node represents the predicted value " $\leq 50K$ " or " $> 50K$ ". During the first running, this model was saved in a file (train_dt.rds) and uploaded to GitHub.
4. Decision Tree - Complexity Parameter: The complexity parameter optimizes the size and complexity of the tree during growth and can improve generalization and avoid overfitting. During the first running, this model was saved in a file (train_dt_cp.rds) and uploaded to GitHub.
5. Random Forest: It randomly generates many decision trees from the same data set to strengthen the prediction. During the first running, this model was saved in a file (train_rf.rds) and uploaded to GitHub.
6. Random Forest with Cross Validation: It applies cross validation to the Random Forest model. During the first running, this model was saved in a file (train_rf_2.rds) and uploaded to GitHub.

2.3.3.1 First model: KNN

The first model applied is KNN (K-Nearest Neighbors), which classifies data points based on their similarity to existing data points. The similarity is measured through the euclidian distance and the minimum distance is chosen to select the neighborhood and predict the values.

The code to load the model from the RDS file is as follows:

```
# KNN =====

Sys.time()      # Time is shown to track duration

## [1] "2025-08-30 18:59:22 CDT"

# Code used to generate model and save to RDS file
#   set.seed(6)
#   train_knn <-   train(income ~ .,
#                         method = "knn",
#                         data = censusTrain,
#                         tuneGrid = data.frame(k = seq(3, 23, 2)))
#   saveRDS(train_knn, "train_knn.rds")

# Loading from file to save running time
train_knn <- readRDS("train_knn.rds")

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-08-30 18:59:22 CDT"
```

The following is the resulting KNN model. It shows that accuracy is maximum at K =

```
## [1] 21

## k-Nearest Neighbors
##
## 34190 samples
##    9 predictor
##    2 classes: '<=50K', '>50K'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 34190, 34190, 34190, 34190, 34190, 34190, ...
## Resampling results across tuning parameters:
##
##    k    Accuracy    Kappa
##    3  0.7807132  0.3987966
##    5  0.7886633  0.4127464
##    7  0.7938105  0.4214002
##    9  0.7967323  0.4250158
##   11  0.7985539  0.4269369
##   13  0.7997944  0.4272276
##   15  0.8004212  0.4265388
##   17  0.8010092  0.4262207
##   19  0.8012845  0.4257372
##   21  0.8014773  0.4245378
##   23  0.8013809  0.4226779
##
## Accuracy was used to select the optimal model using the largest value.
```

The final value used for the model was $k = 21$.

Following is the Variables Importance plot for the KNN model. This graph shows how much each predictor variable contributes to the model's predictive power.

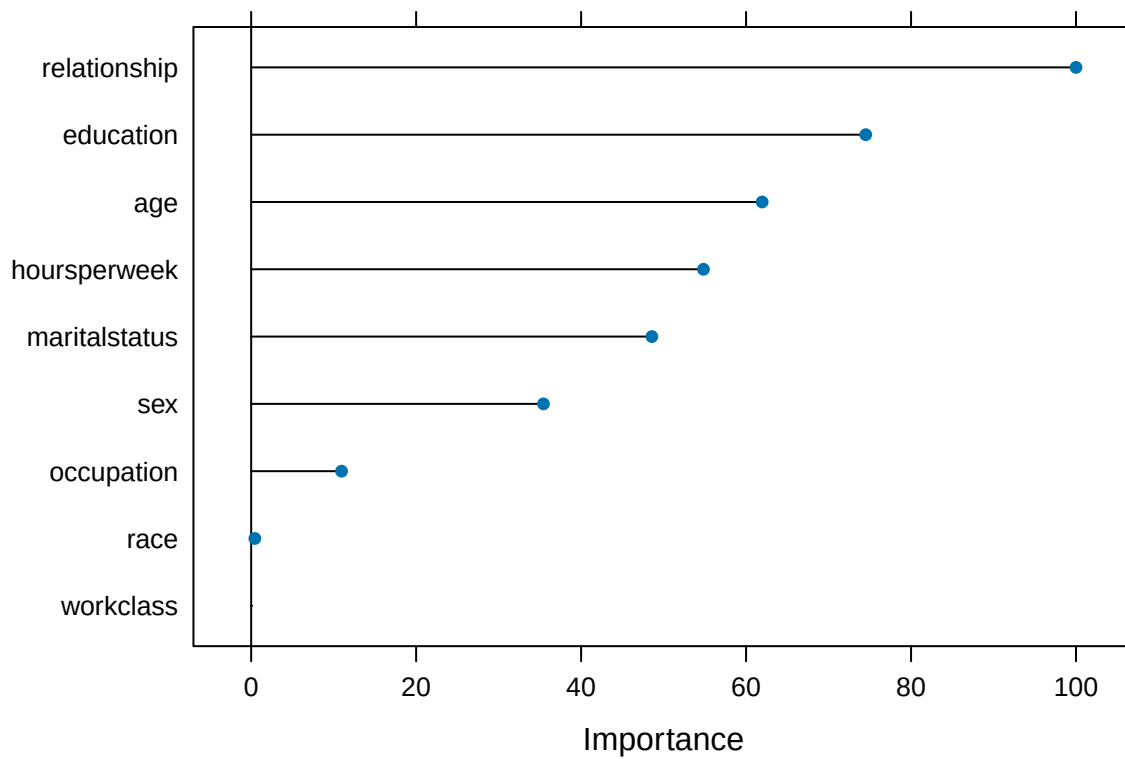
It shows that :

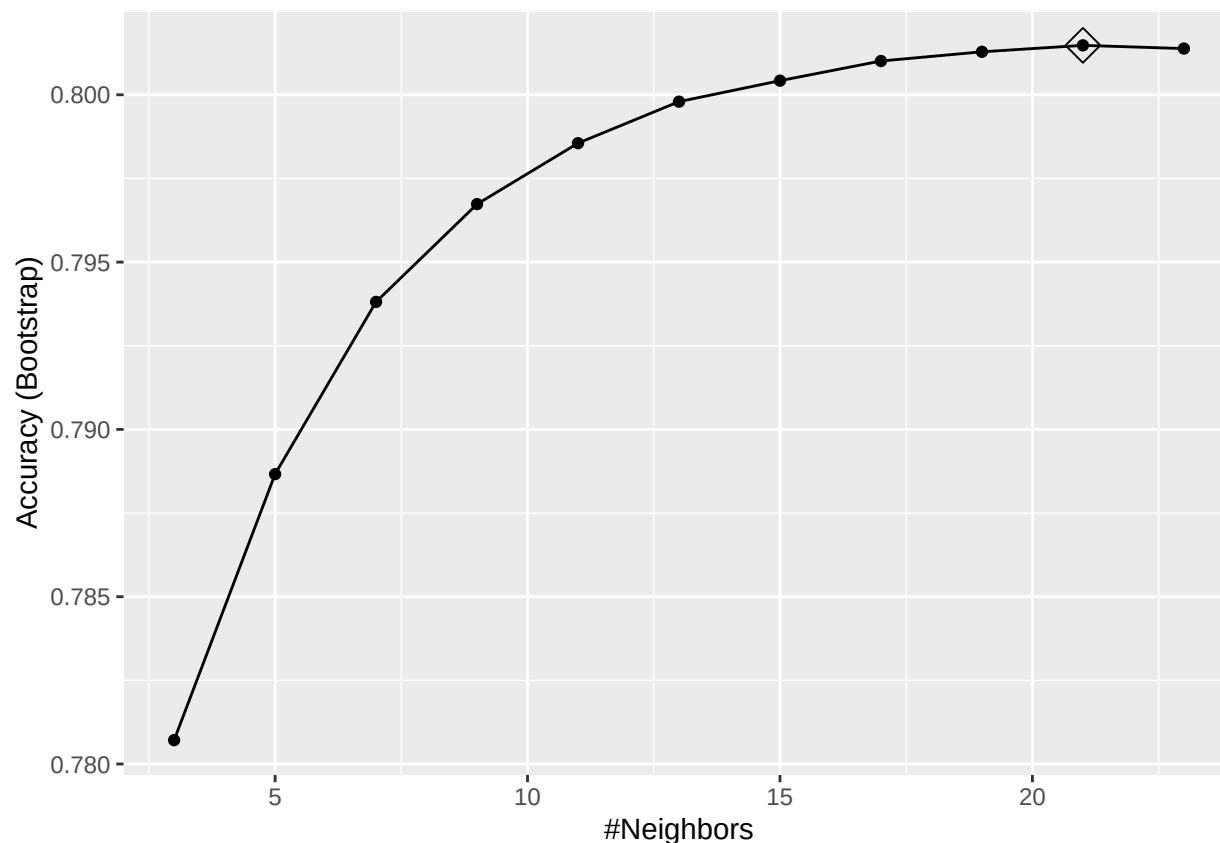
- relationship
- education
- age

Are the most important variables for this model.

Next is a plot of Accuracy versus K showing that Accuracy is maximized at $K =$

[1] 21





The following section shows the Confusion Matrix for the Train set. Among other parameters, it shows:

- Accuracy
- Sensitivity
- Specificity
- Balanced Accuracy

```
## Accuracy: 82.16 %
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##    <=50K 23757 3847
##    >50K  2252 4334
##
##           Accuracy : 0.8216
##           95% CI : (0.8175, 0.8257)
##    No Information Rate : 0.7607
##    P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.4749
##
##    Mcnemar's Test P-Value : < 2.2e-16
##
##           Sensitivity : 0.9134
##           Specificity : 0.5298
```

```

##          Pos Pred Value : 0.8606
##          Neg Pred Value : 0.6581
##          Prevalence : 0.7607
##          Detection Rate : 0.6949
##    Detection Prevalence : 0.8074
##          Balanced Accuracy : 0.7216
##
##          'Positive' Class : <=50K
##

```

The First model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8216145	K = 21

2.3.3.2 Second model: KNN with Cross Validation

KNN with Cross Validation is a model that uses tuning to find the optimum value of K.

The code to load the model from the RDS file is as follows:

```
# KNN Cross Validation =====

Sys.time()      # Time is shown to track duration

## [1] "2025-08-30 19:01:51 CDT"

# Code used to generate model and save to RDS file
# control <- trainControl(method = "cv", number = 5, p = .9)
# set.seed(6)
# train_knn_cv <- train(income ~ ., method = "knn",
#                       data = censusTrain,
#                       tuneGrid = data.frame(k = seq(3, 31, 2)),
#                       trControl = control)
# saveRDS(train_knn_cv, "train_knn_cv.rds")

# Loading model from file to save running time
train_knn_cv <- readRDS("train_knn_cv.rds")

Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-08-30 19:01:51 CDT"
```

The following is the resulting KNN_CV model. It shows that accuracy is maximum at K =

```
## [1] 15

## k-Nearest Neighbors
##
## 34190 samples
##    9 predictor
##    2 classes: '<=50K', '>50K'
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 27352, 27352, 27352, 27352, 27352
## Resampling results across tuning parameters:
##
##  k  Accuracy  Kappa
##  3  0.7970459  0.4338884
##  5  0.8042410  0.4460149
##  7  0.8054694  0.4438641
##  9  0.8049430  0.4403165
## 11  0.8044165  0.4355521
## 13  0.8048260  0.4348288
## 15  0.8056742  0.4349668
## 17  0.8048260  0.4304711
## 19  0.8049722  0.4290164
## 21  0.8053524  0.4292203
## 23  0.8055572  0.4286341
## 25  0.8052354  0.4263177
## 27  0.8043580  0.4222083
## 29  0.8040363  0.4201429
```

```
## 31 0.8042703 0.4201323
```

```
##
```

```
## Accuracy was used to select the optimal model using the largest value.
```

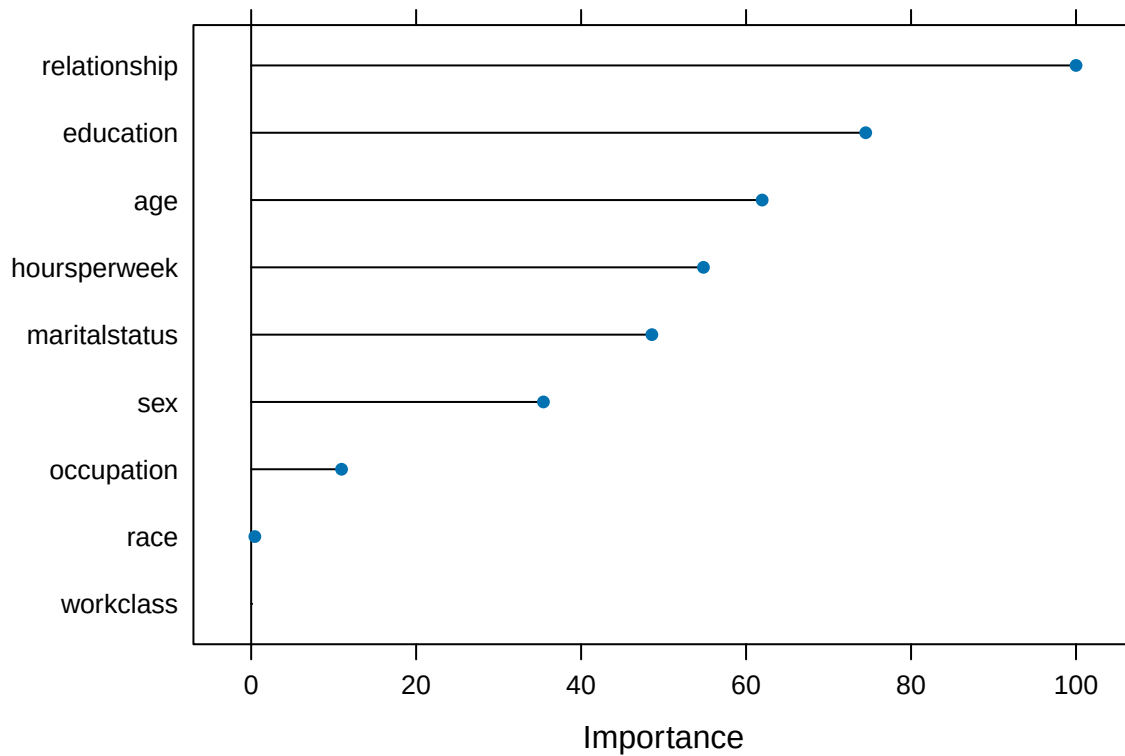
```
## The final value used for the model was k = 15.
```

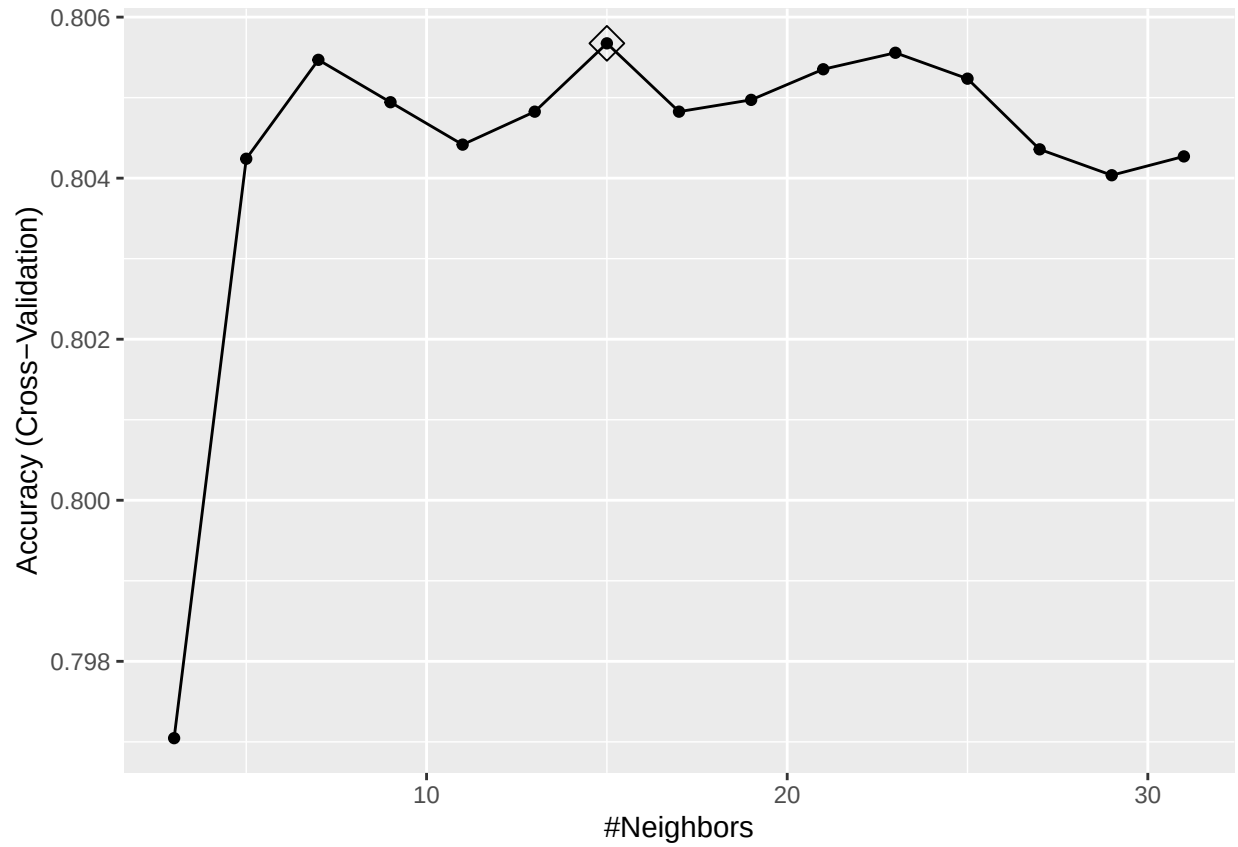
Following is the Variables Importance plot for the KNN_CV model. This graph shows how much each predictor variable contributes to the model's predictive power.

It shows that :

- relationship
- education
- age

Are the most important variables for this model.

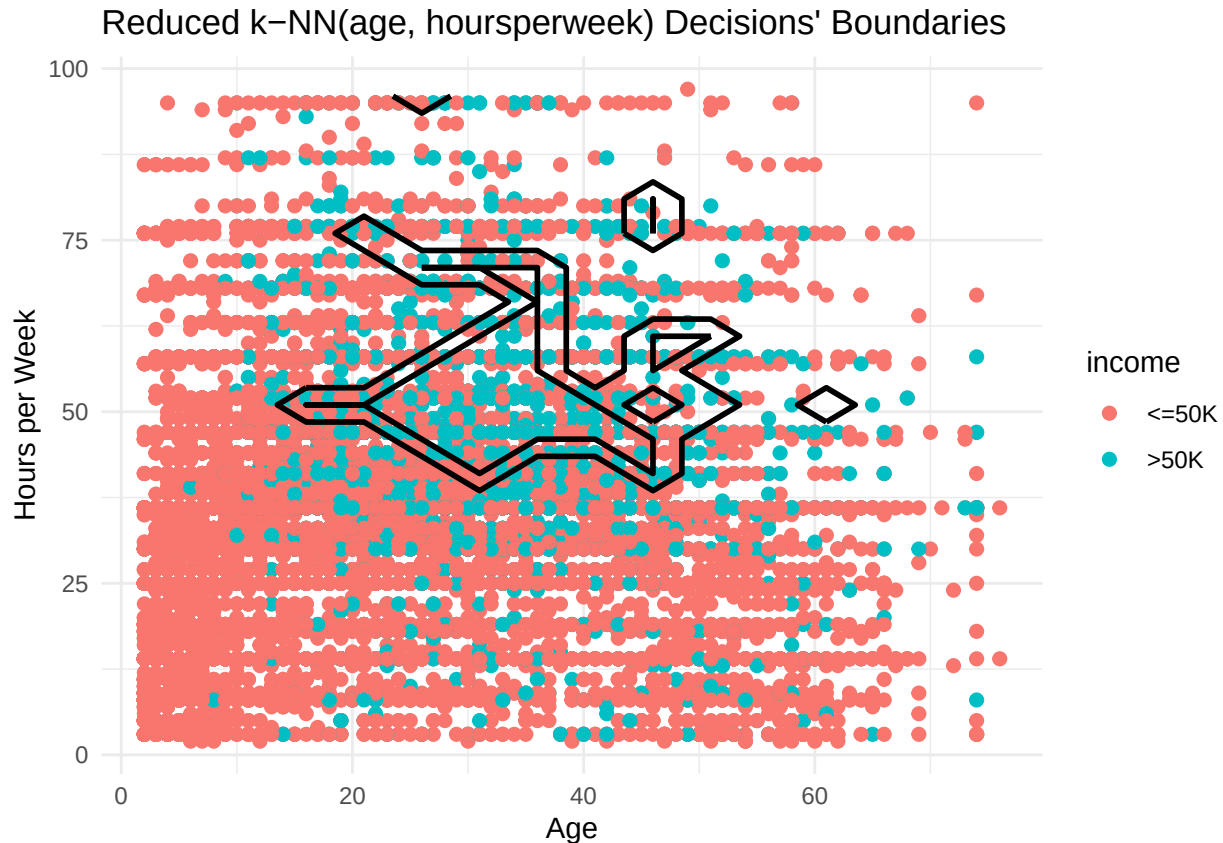




The previous plot shows Accuracy versus K showing that Accuracy is maximized at K =

[1] 15

The following plot is illustrative of the behavior of the KNN CV model. Because there are 9 predictors, they cannot be plotted in just one graph with two dimensions. A reduced KNN CV model was generated with just the two most important continuous variables for KNN: “Age” and “Hours per Week”. The contour of the reduced model’s predictions was then plotted using CensusTrain. The contour shows the decisions’ boundaries to assign “<=50K” or “>50K”



The following section shows the Confusion Matrix for the Train set. Among other parameters, it shows:

- Accuracy
- Sensitivity
- Specificity
- Balanced Accuracy

```
## Accuracy:  82.5 %
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##      <=50K 23707 3680
##      >50K  2302 4501
##
##           Accuracy : 0.825
##           95% CI : (0.821, 0.8291)
##      No Information Rate : 0.7607
##      P-Value [Acc > NIR] : < 2.2e-16
##
```

```

##                Kappa : 0.49
##
## Mcnemar's Test P-Value : < 2.2e-16
##
##          Sensitivity : 0.9115
##          Specificity : 0.5502
##          Pos Pred Value : 0.8656
##          Neg Pred Value : 0.6616
##          Prevalence : 0.7607
##          Detection Rate : 0.6934
##          Detection Prevalence : 0.8010
##          Balanced Accuracy : 0.7308
##
##          'Positive' Class : <=50K
##

```

The second model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8216145	K = 21
2: KNN with CV (Cross Validation)	0.8250366	K = 15

2.3.3.3 Third model: Decision Tree

Decision Tree splits data into subsets based on a series of conditions identified based on minimizing a coefficient. Each internal node represents a test (Yes or No) on a predictor, each branch represents the outcome of the test, and each leaf node represents the predicted value “ $\leq 50K$ ” or “ $> 50K$ ”.

The code to load the model from the RDS file is as follows:

```
# DECISION TREE =====

Sys.time()      # Time is shown to track duration

## [1] "2025-08-30 19:04:22 CDT"

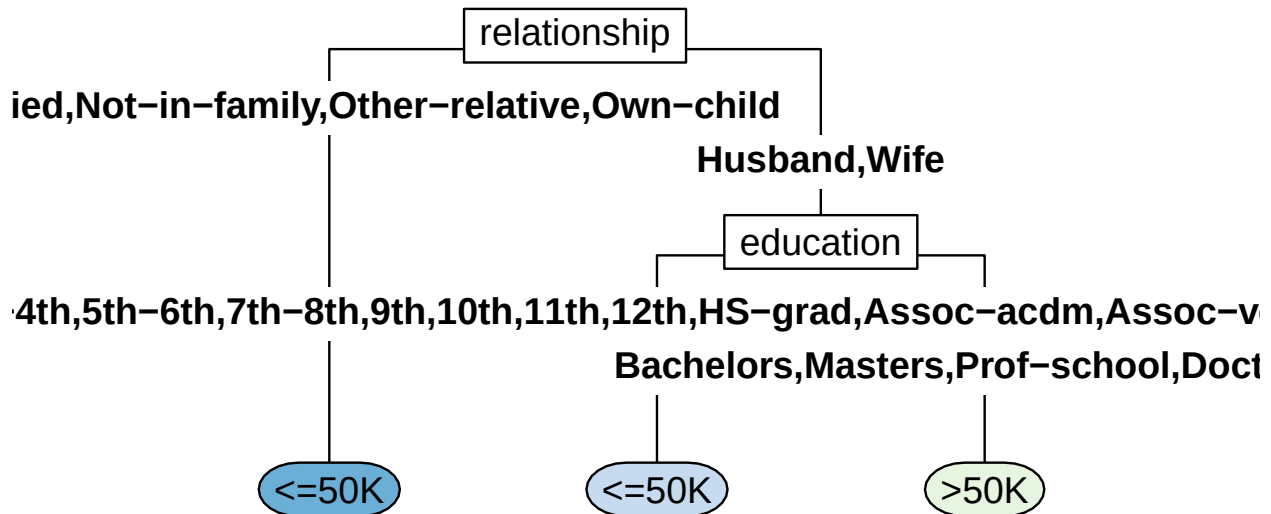
# Code used to generate model and save to RDS file
#   set.seed(6)
#   train_dt <- rpart(income ~ ., data = censusTrain)
#   saveRDS(train_dt, "train_dt.rds")

# Loading model from file to save running time
train_dt <- readRDS("train_dt.rds")

Sys.time()      # Time is shown to track duration

## [1] "2025-08-30 19:04:22 CDT"
```

Next is the plot of the Decision Tree model followed by the rules of the model and its Accuracy.



```

## n= 34190
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 34190 8181 <=50K (0.76071951 0.23928049)
##    2) relationship=Unmarried,Not-in-family,Other-relative,Own-child 18795 1245 <=50K (0.93375898 0.06624102)
##    3) relationship=Husband,Wife 15395 6936 <=50K (0.54946411 0.45053589)
##      6) education=Preschool,1st-4th,5th-6th,7th-8th,9th,10th,11th,12th,HS-grad,Assoc-acdm,Assoc-voc,Some-college 11420 5710 <=50K (0.54946411 0.45053589)
##      7) education=Bachelors,Masters,Prof-school,Doctorate 4620 1295 >50K (0.28030303 0.71969697) *

```

Following is the Variables Importance list for the Decision Tree model. This list shows how much each predictor variable contributes to the model's predictive power.

It shows that :

- relationship
- maritalstatus
- education

Are the most important variables for this model.

	Overall
relationship	2499.6666
maritalstatus	2472.1641
education	2262.5840
occupation	2052.8700
age	1219.4816
hoursperweek	194.9639
workclass	122.3880
race	0.0000
sex	0.0000

The following section shows the Confusion Matrix for the Train set. Among other parameters, it shows:

- Accuracy
- Sensitivity
- Specificity
- Balanced Accuracy

```
## Accuracy: 82.12 %
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##    <=50K 24716 4821
##    >50K 1293 3360
##
##           Accuracy : 0.8212
##           95% CI : (0.8171, 0.8252)
##    No Information Rate : 0.7607
##    P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.4236
##
## Mcnemar's Test P-Value : < 2.2e-16
##
##           Sensitivity : 0.9503
##           Specificity : 0.4107
##           Pos Pred Value : 0.8368
##           Neg Pred Value : 0.7221
##           Prevalence : 0.7607
##           Detection Rate : 0.7229
##           Detection Prevalence : 0.8639
##           Balanced Accuracy : 0.6805
##
##           'Positive' Class : <=50K
##
```

The third model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8216145	K = 21
2: KNN with CV (Cross Validation)	0.8250366	K = 15

method	Accuracy	Parameter
3: Decision tree	0.8211758	

2.3.3.4 Fourth model: Decision Tree with Complexity Parameter

The complexity parameter in a Decision Tree optimizes the size and complexity of the tree during growth and can improve generalization and avoid overfitting.

The code to load the model from the RDS file is as follows:

```
# Decision Tree with Complexity Parameter (CP) =====  
  
Sys.time()      # Time is shown to track duration  
  
## [1] "2025-08-30 19:04:23 CDT"  
  
# Code used to generate model and save to RDS file  
#   set.seed(6)  
#   train_dt_cp <- train(income ~ .,  
#                         method = "rpart",  
#                         tuneGrid = data.frame(cp = seq(0.0, 0.1, len = 25)),  
#                         data = censusTrain)  
#   saveRDS(train_dt_cp, "train_dt_cp.rds")  
  
# Loading model from file to save running time  
train_dt_cp <- readRDS("train_dt_cp.rds")  
  
Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-08-30 19:04:25 CDT"
```

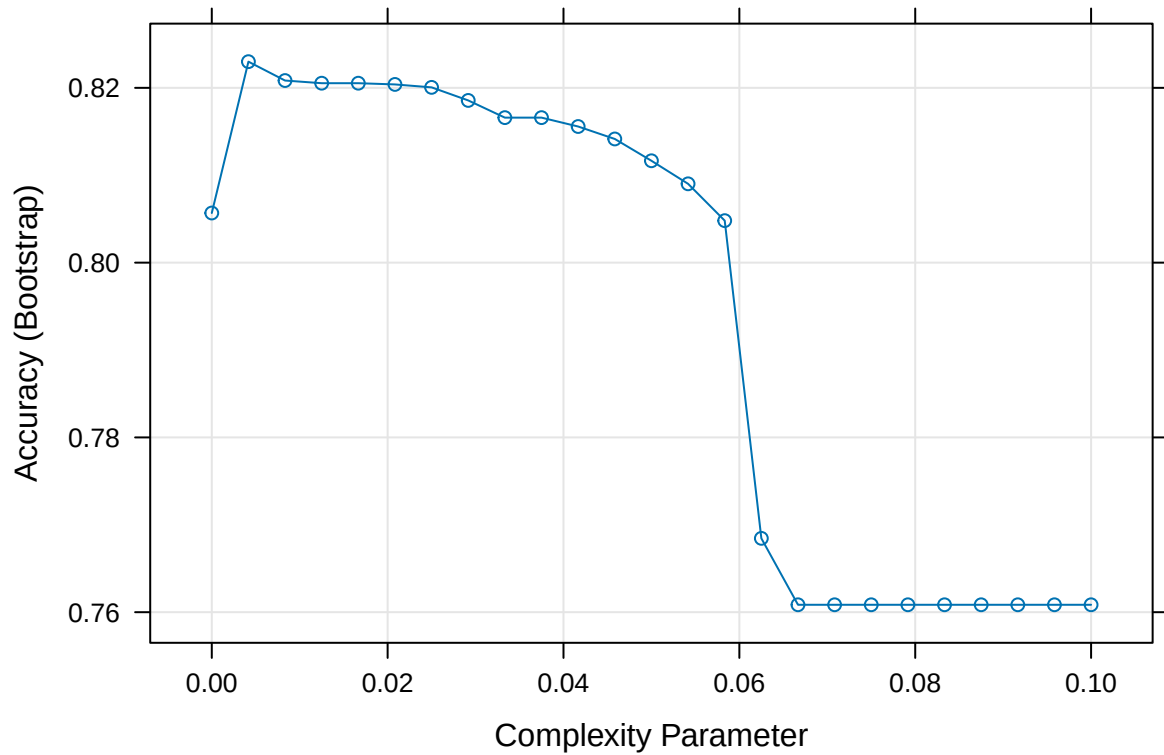
Next is the Decision Tree model and the plot of Complexity Parameter versus Accuracy. It shows that Accuracy is maximum for a CP of:

```
## [1] 0.004166667  
  
## CART  
##  
## 34190 samples  
##    9 predictor  
##    2 classes: '<=50K', '>50K'  
##  
## No pre-processing  
## Resampling: Bootstrapped (25 reps)  
## Summary of sample sizes: 34190, 34190, 34190, 34190, 34190, 34190, ...  
## Resampling results across tuning parameters:  
##  
##    cp          Accuracy    Kappa  
## 0.000000000 0.8056779 0.45059127  
## 0.004166667 0.8229993 0.45706671  
## 0.008333333 0.8208381 0.45815896  
## 0.012500000 0.8205329 0.45856803  
## 0.016666667 0.8205329 0.45856803  
## 0.020833333 0.8203927 0.45686008  
## 0.025000000 0.8200558 0.45441134  
## 0.029166667 0.8185618 0.44272626  
## 0.033333333 0.8165925 0.42695830  
## 0.037500000 0.8165925 0.42695830  
## 0.041666667 0.8155758 0.42023598  
## 0.045833333 0.8141528 0.41004991
```

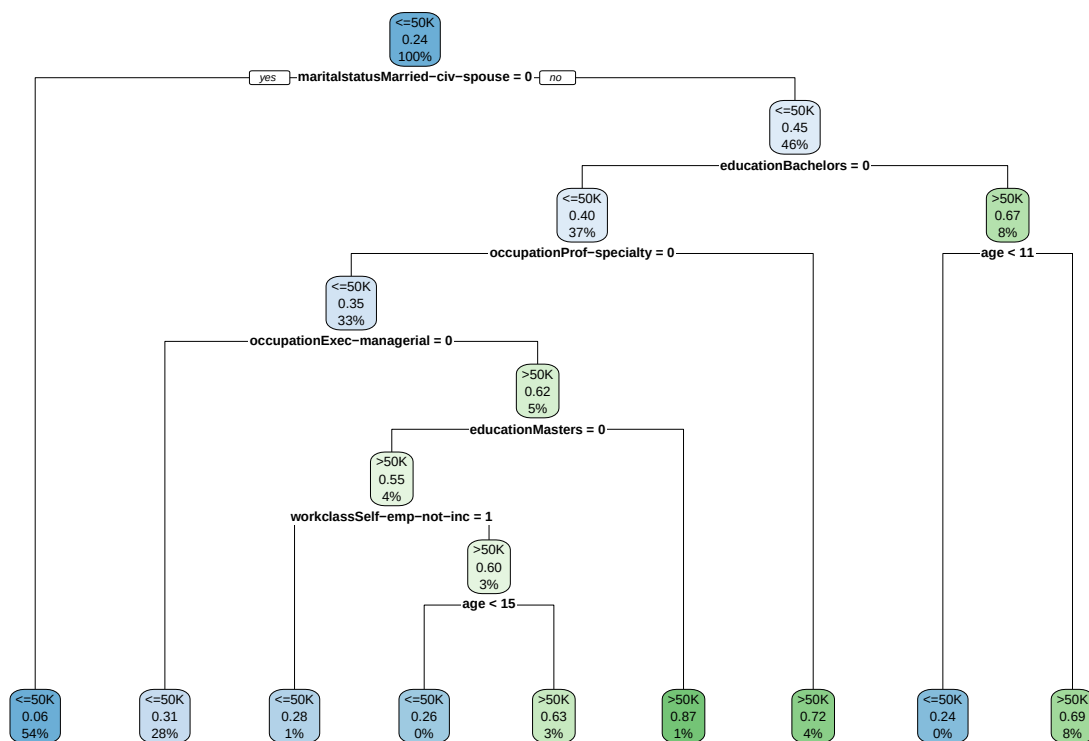
```

## 0.050000000 0.8116562 0.39360342
## 0.054166667 0.8090199 0.37416896
## 0.058333333 0.8048036 0.34182816
## 0.062500000 0.7684515 0.05907611
## 0.066666667 0.7608491 0.00000000
## 0.070833333 0.7608491 0.00000000
## 0.075000000 0.7608491 0.00000000
## 0.079166667 0.7608491 0.00000000
## 0.083333333 0.7608491 0.00000000
## 0.087500000 0.7608491 0.00000000
## 0.091666667 0.7608491 0.00000000
## 0.095833333 0.7608491 0.00000000
## 0.100000000 0.7608491 0.00000000
##
## Accuracy was used to select the optimal model using the largest value.
## The final value used for the model was cp = 0.004166667.

```



Following are the plot of the Decision Tree model with Complexity Parameter and the description of its rules.

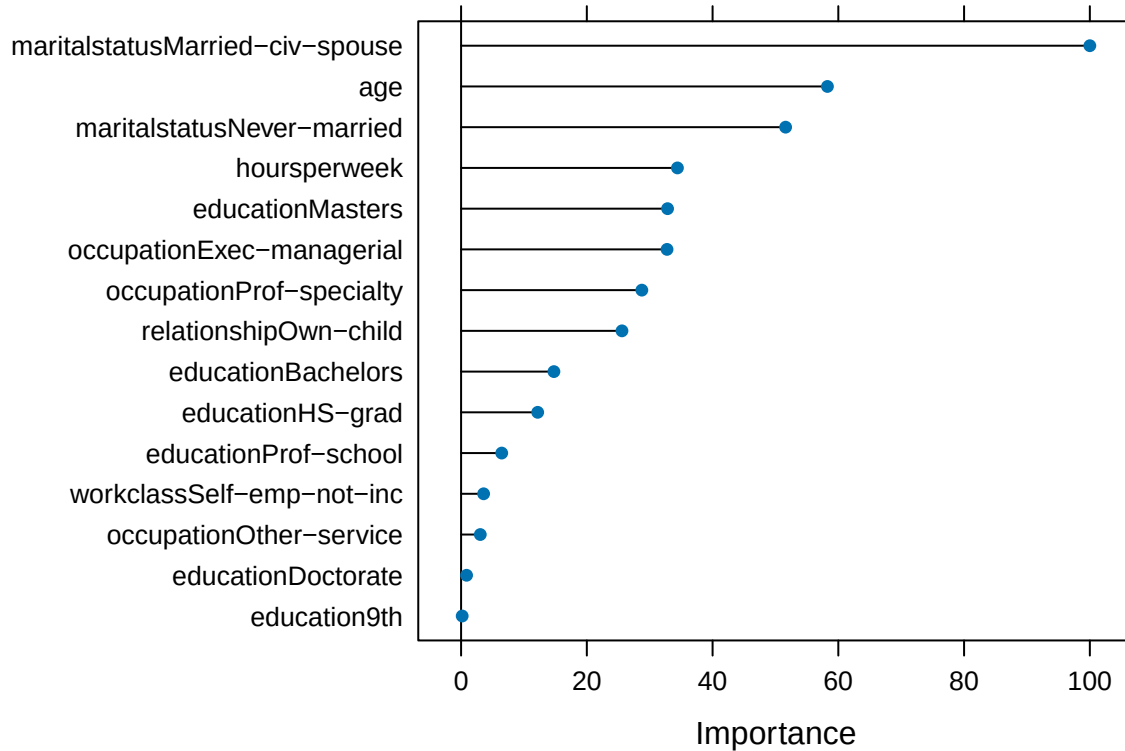


```

## n= 34190
##
## node), split, n, loss, yval, (yprob)
##      * denotes terminal node
##
## 1) root 34190 8181 <=50K (0.76071951 0.23928049)
##    2) maritalstatusMarried-civ-spouse< 0.5 18573 1207 <=50K (0.93501319 0.06498681) *
##    3) maritalstatusMarried-civ-spouse>=0.5 15617 6974 <=50K (0.55343536 0.44656464)
##    6) educationBachelors< 0.5 12727 5028 <=50K (0.60493439 0.39506561)
##    12) occupationProf-specialty< 0.5 11236 3959 <=50K (0.64765041 0.35234959)
##    24) occupationExec-managerial< 0.5 9592 2936 <=50K (0.69391159 0.30608841) *
##    25) occupationExec-managerial>=0.5 1644 621 >50K (0.37773723 0.62226277)
##    50) educationMasters< 0.5 1292 576 >50K (0.44582043 0.55417957)
##    100) workclassSelf-emp-not-inc>=0.5 189 53 <=50K (0.71957672 0.28042328) *
##    101) workclassSelf-emp-not-inc< 0.5 1103 440 >50K (0.39891206 0.60108794)
##    202) age< 14.5 77 20 <=50K (0.74025974 0.25974026) *
##    203) age>=14.5 1026 383 >50K (0.37329435 0.62670565) *
##    51) educationMasters>=0.5 352 45 >50K (0.12784091 0.87215909) *
##    13) occupationProf-specialty>=0.5 1491 422 >50K (0.28303152 0.71696848) *
## 7) educationBachelors>=0.5 2890 944 >50K (0.32664360 0.67335640)
##    14) age< 10.5 78 19 <=50K (0.75641026 0.24358974) *
##    15) age>=10.5 2812 885 >50K (0.31472262 0.68527738) *

```

The Variables Importance plot is shown for the Decision Tree model. This plot shows how much each variable and Values contribute to the model's predictive power.



The following section shows the Confusion Matrix for the Train set. Among other parameters, it shows:

- Accuracy
- Sensitivity
- Specificity
- Balanced Accuracy

```
## Accuracy: 82.71 %
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##    <=50K  24308  4210
##    >50K   1701  3971
##
##           Accuracy : 0.8271
##           95% CI : (0.8231, 0.8311)
##    No Information Rate : 0.7607
##    P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.4693
##
```

```

## McNemar's Test P-Value : < 2.2e-16
##
##          Sensitivity : 0.9346
##          Specificity : 0.4854
##          Pos Pred Value : 0.8524
##          Neg Pred Value : 0.7001
##          Prevalence : 0.7607
##          Detection Rate : 0.7110
##          Detection Prevalence : 0.8341
##          Balanced Accuracy : 0.7100
##
##          'Positive' Class : <=50K
##

```

The fourth model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8216145	K = 21
2: KNN with CV (Cross Validation)	0.8250366	K = 15
3: Decision tree	0.8211758	
4: Decision tree with CP (Complexity Parameter)	0.8271132	cp = 0.004166666666666667

2.3.3.5 Fifth model: Random Forest

Random Forest generates many decision trees from the same data set to strengthen the prediction.

The code to load the model from the RDS file is as follows:

```
# Random Forest =====
```

```
Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-08-30 19:04:26 CDT"
```

```
# Code used to generate model and save to RDS file
```

```
# set.seed(6)
```

```
# train_rf<-randomForest(formula=income~.,
```

```
#                        data=censusTrain,
```

```
#                        ntree=500,
```

```
#                        do.trace=TRUE)
```

```
# saveRDS(train_rf,"train_rf.rds")
```

```
# Loading model from file to save running time
```

```
train_rf <- readRDS("train_rf.rds")
```

```
Sys.time()      # Time is shown to track duration
```

```
## [1] "2025-08-30 19:04:27 CDT"
```

Next is the Random Forest model using 500 trees with 3 variables tried at each split.

```
##
```

```
## Call:
```

```
## randomForest(formula = income ~ ., data = censusTrain, ntree = 500,      do.trace = TRUE)
```

```
##           Type of random forest: classification
```

```
##           Number of trees: 500
```

```
## No. of variables tried at each split: 3
```

```
##
```

```
##           OOB estimate of  error rate: 16.33%
```

```
## Confusion matrix:
```

```
##           <=50K >50K class.error
```

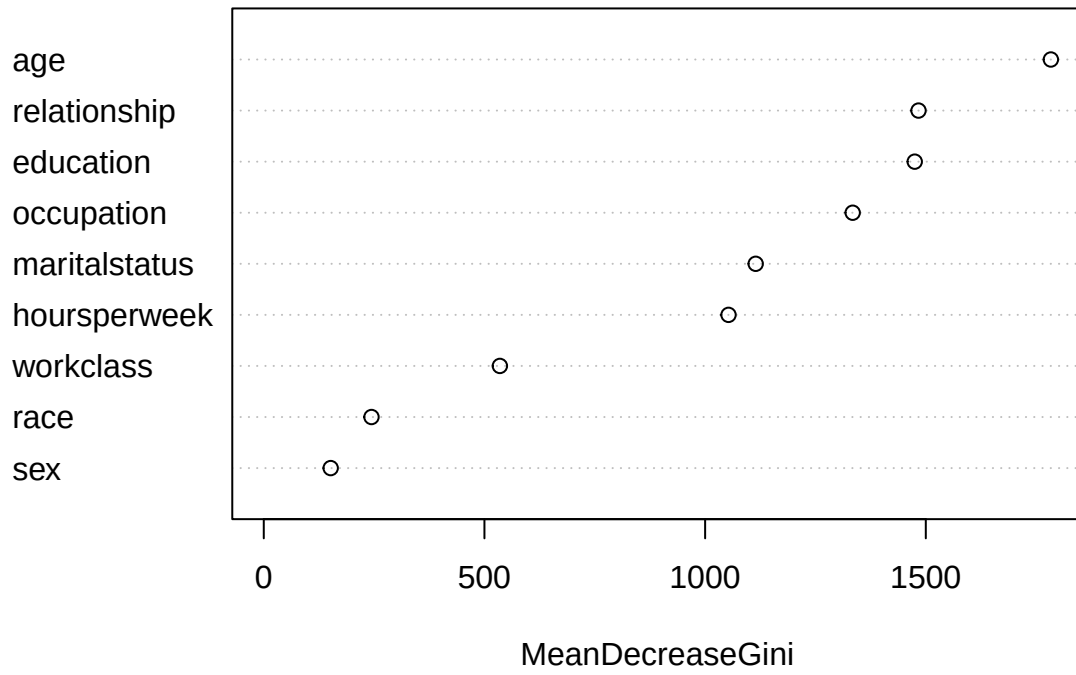
```
## <=50K  23861  2148   0.0825868
```

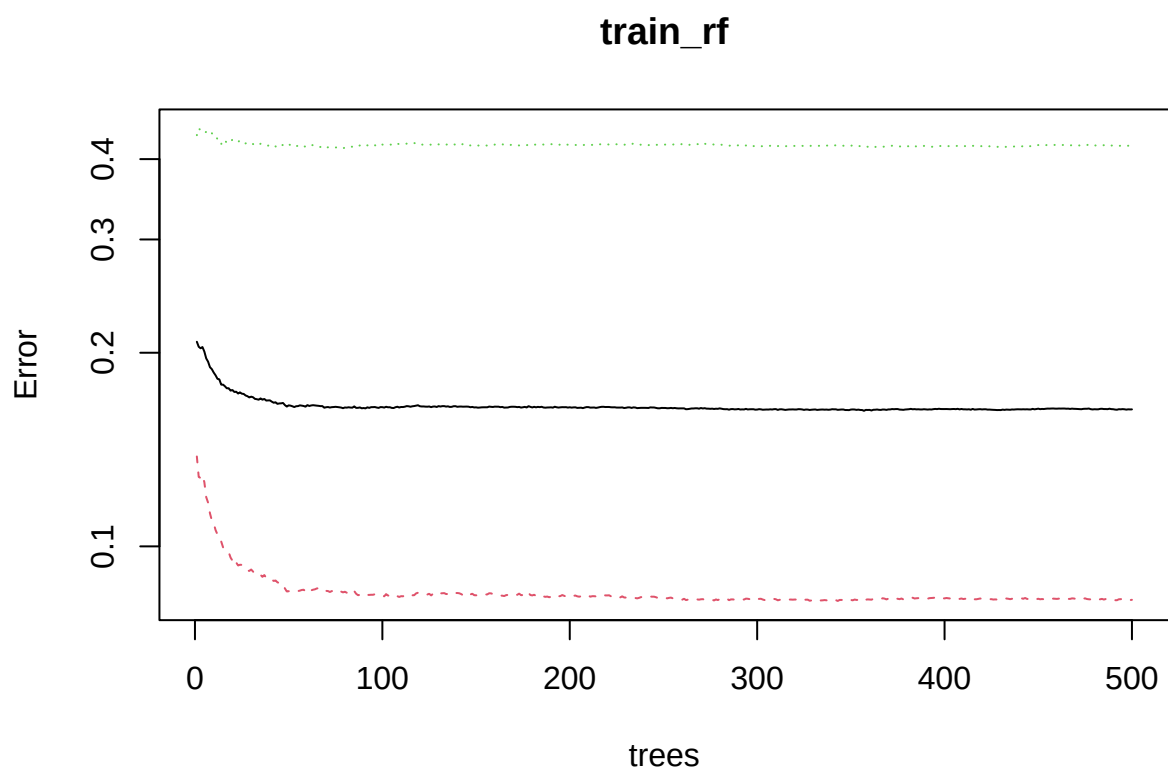
```
## >50K   3435  4746   0.4198753
```

The variables Importance plot is shown for the Random Forest model. This plot shows how much each variable contributes to the models's predictive power. It shows that Age, Relationship and Education are the most important variables for the model. This is also reflected in the plot of Variables versus MeanDecreaseGini that follows.

	Overall
age	1783.3898
relationship	1483.5272
education	1474.9018
occupation	1334.3591
maritalstatus	1114.5566
hoursperweek	1053.0452
workclass	534.9586
race	244.0581
sex	151.6017

train_rf

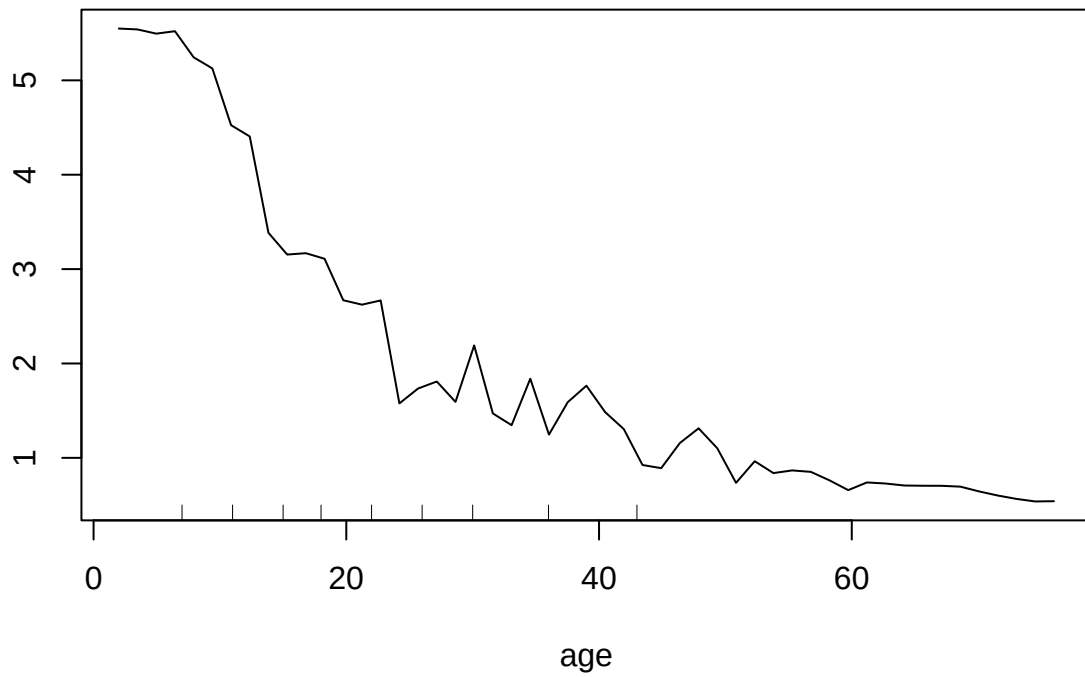




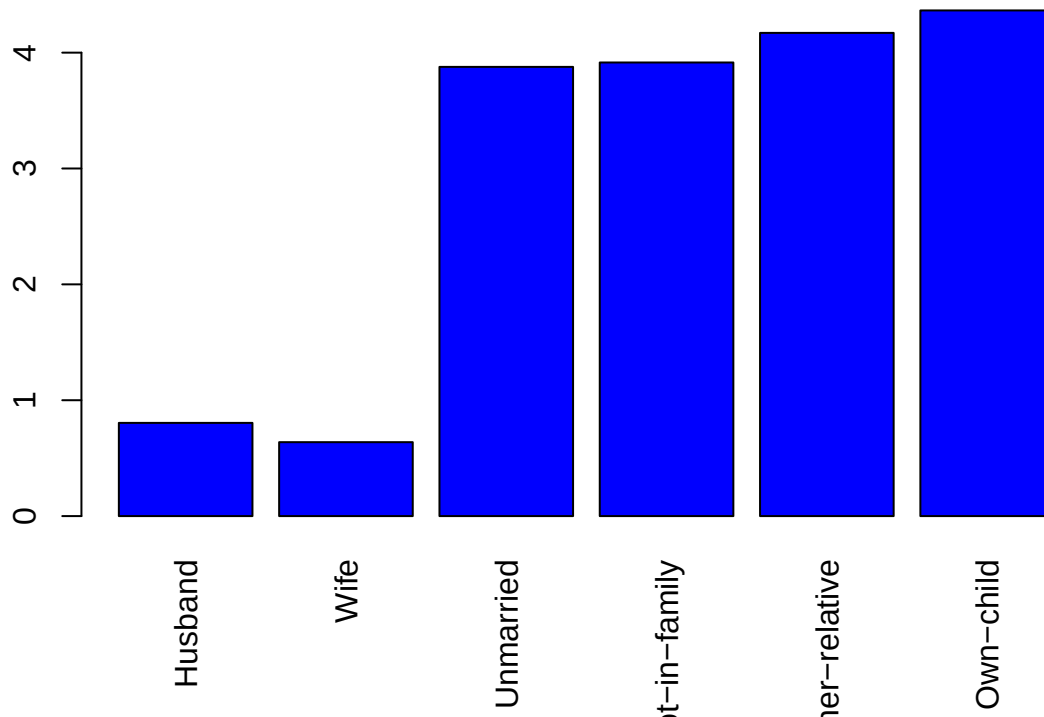
The previous plot shows how error is decreasing as number of trees is increasing

The following two graphs show the dependance of the model on the values of the two most important variables: age and relationship.

Partial Dependence on age



Partial Dependence on relationship



It shows that the model is more dependant for Ages less than 25 and Relationship as Husband and Wife.

The following section shows the Confusion Matrix for the Train set. Among other parameters, it shows:

- Accuracy
- Sensitivity
- Specificity
- Balanced Accuracy

```
## Accuracy:  92.68 %
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##    <=50K  25024  1518
##    >50K    985  6663
##
##           Accuracy : 0.9268
##           95% CI   : (0.924, 0.9295)
##    No Information Rate : 0.7607
##    P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa   : 0.7943
##
##    McNemar's Test P-Value : < 2.2e-16
##
```

```

##          Sensitivity : 0.9621
##          Specificity : 0.8144
##          Pos Pred Value : 0.9428
##          Neg Pred Value : 0.8712
##          Prevalence : 0.7607
##          Detection Rate : 0.7319
##          Detection Prevalence : 0.7763
##          Balanced Accuracy : 0.8883
##
##          'Positive' Class : <=50K
##

```

The fifth model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8216145	K = 21
2: KNN with CV (Cross Validation)	0.8250366	K = 15
3: Decision tree	0.8211758	
4: Decision tree with CP (Complexity Parameter)	0.8271132	cp = 0.004166666666666667
5: Random forest	0.9267915	n of trees = 500, n of var = 3

2.3.3.6 Sixth model: Random Forest with Cross Validation

Cross validation is applied to the Random Forest model.

The code to load the model from the RDS file is as follows:

```
# Random Forest with CV =====

Sys.time()      # Time is shown to track duration

## [1] "2025-08-30 19:07:45 CDT"

# Code used to generate model and save to RDS file
#   set.seed(6)
#   train_rf_2 <- train(income ~ .,
#                       method = "Rborist",
#                       tuneGrid = data.frame(predFixed = 2, minNode = c(3, 50)),
#                       data = censusTrain)
#   saveRDS(train_rf_2, "train_rf_2.rds")

# Loading model from file to save running time
train_rf_2 <- readRDS("train_rf_2.rds")

Sys.time()      # Time is shown to track duration

## [1] "2025-08-30 19:07:46 CDT"
```

Next is the Random Forest model with Cross Validation generated using 300 trees with 3 variables tried at each split.

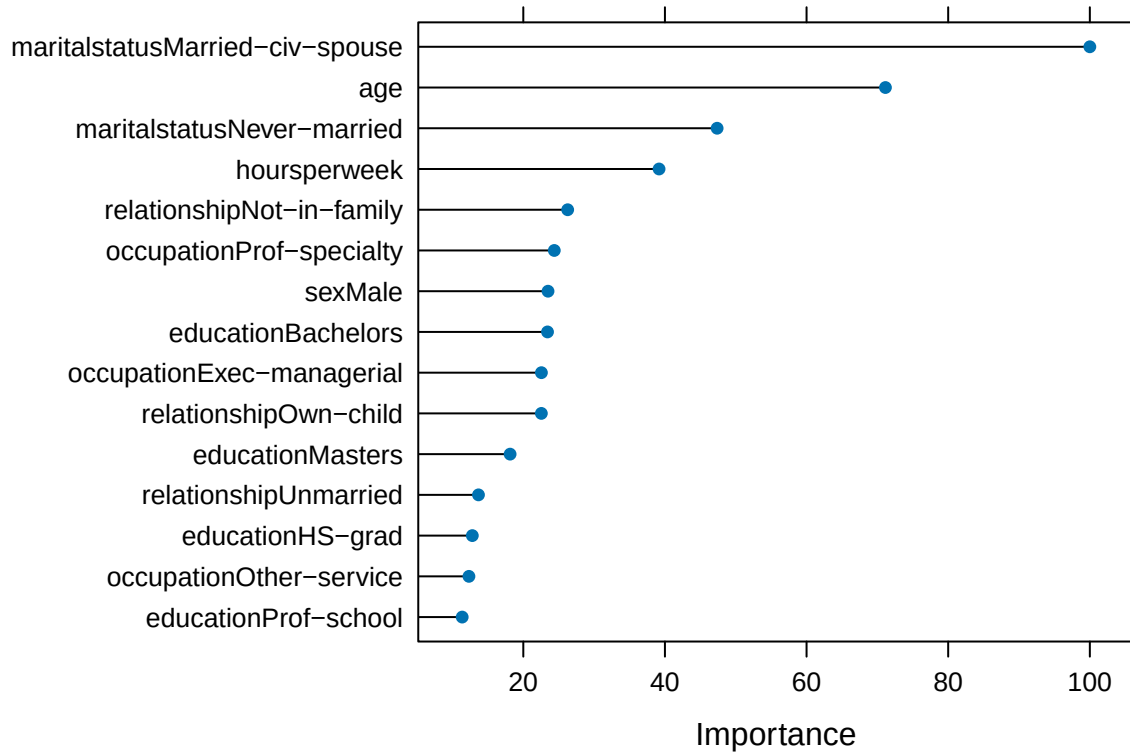
```
## Random Forest
##
## 34190 samples
##    9 predictor
##    2 classes: '<=50K', '>50K'
##
## No pre-processing
## Resampling: Bootstrapped (25 reps)
## Summary of sample sizes: 34190, 34190, 34190, 34190, 34190, 34190, ...
## Resampling results across tuning parameters:
##
##  minNode  Accuracy  Kappa
##    3      0.8149345  0.3487248
##   50      0.8119857  0.3306690
##
## Tuning parameter 'predFixed' was held constant at a value of 2
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were predFixed = 2 and minNode = 3.
```

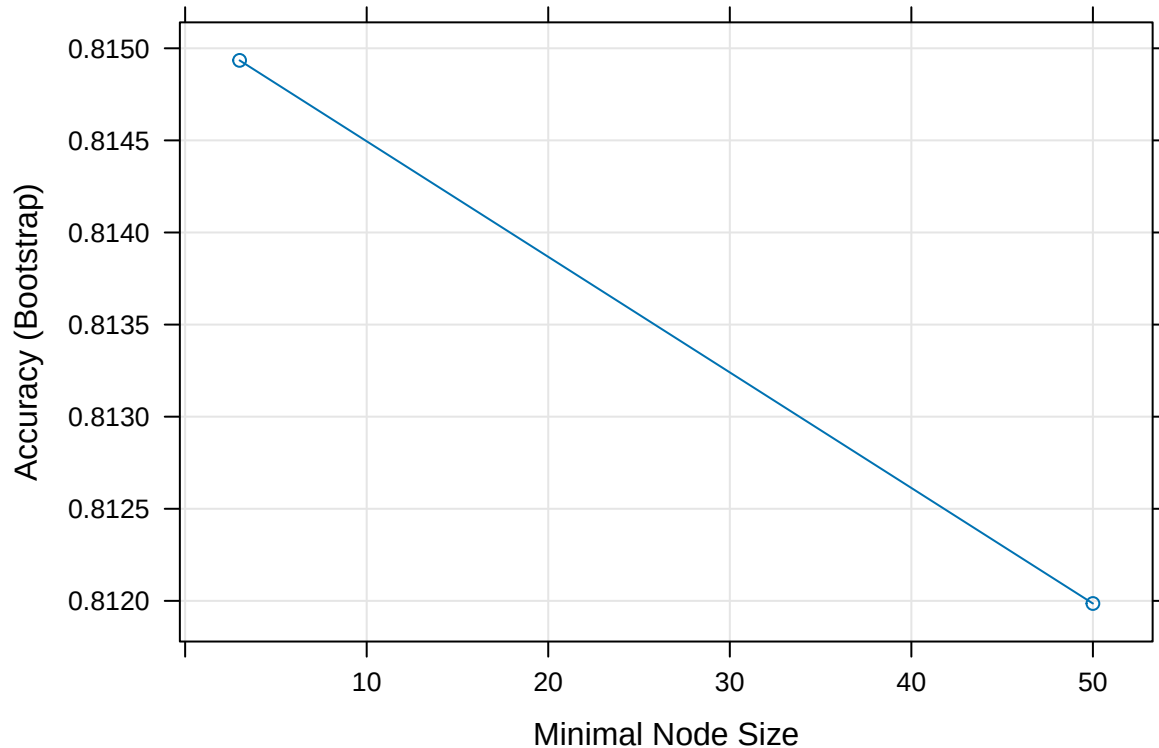
The Variables Importance plot is shown for the Random Forest CV model. This plot shows how much each variable contributes to the model's predictive power.

It shows that:

- MaritalStatus-Married-Civ-Spouse
- Age
- MaritalStatus-Never-married

Are the most important variables for this model.





The previous plot shows how Accuracy decreases as Minimal Node Size increases.

The following section shows the Confusion Matrix for the Train set. Among other parameters, it shows:

- Accuracy
- Sensitivity
- Specificity
- Balanced Accuracy

```
## Accuracy: 81.57 %
## Confusion Matrix and Statistics
##
##           Reference
## Prediction <=50K >50K
##    <=50K 25634 5926
##    >50K   375 2255
##
##           Accuracy : 0.8157
##           95% CI : (0.8116, 0.8198)
##    No Information Rate : 0.7607
##    P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.3404
##
## Mcnemar's Test P-Value : < 2.2e-16
##
##           Sensitivity : 0.9856
```

```

##          Specificity : 0.2756
##          Pos Pred Value : 0.8122
##          Neg Pred Value : 0.8574
##          Prevalence : 0.7607
##          Detection Rate : 0.7498
##          Detection Prevalence : 0.9231
##          Balanced Accuracy : 0.6306
##
##          'Positive' Class : <=50K
##

```

The sixth model and its Accuracy are as follows:

method	Accuracy	Parameter
1: KNN	0.8216145	K = 21
2: KNN with CV (Cross Validation)	0.8250366	K = 15
3: Decision tree	0.8211758	
4: Decision tree with CP (Complexity Parameter)	0.8271132	cp = 0.004166666666666667
5: Random forest	0.9267915	n of trees = 500, n of var = 3
6: Random forest with CV (Cross Valdidation)	0.8157063	

3 RESULTS

The following table shows all six models and their Accuracies:

method	Accuracy	Parameter
1: KNN	0.8216145	K = 21
2: KNN with CV (Cross Validation)	0.8250366	K = 15
3: Decision tree	0.8211758	
4: Decision tree with CP (Complexity Parameter)	0.8271132	cp = 0.004166666666666667
5: Random forest	0.9267915	n of trees = 500, n of var = 3
6: Random forest with CV (Cross Valddiation)	0.8157063	

Accuracy has been improving when incorporating Cross Validation or the Complexity Parameter to the base models.

The Random Forest model optimized Accuracy among the six models.

With the Random Forest model, an Income prediction model was built with an Accuracy on the Train dataset of:

```
## Accuracy: 92.68 %
```

Now, the selected model (Random Forest) is applied to the Test dataset and its accuracy calculated as follows:

```
## Accuracy: 83.63 %
```

```
## Confusion Matrix and Statistics
```

```
##
```

```
##           Reference
```

```
## Prediction <=50K >50K
```

```
##           <=50K 10249 1502
```

```
##           >50K   897 2004
```

```
##
```

```
##           Accuracy : 0.8363
```

```
##           95% CI : (0.8302, 0.8422)
```

```
##           No Information Rate : 0.7607
```

```
##           P-Value [Acc > NIR] : < 2.2e-16
```

```
##
```

```
##           Kappa : 0.522
```

```
##
```

```
## McNemar's Test P-Value : < 2.2e-16
```

```
##
```

```
##           Sensitivity : 0.9195
```

```
##           Specificity : 0.5716
```

```
##           Pos Pred Value : 0.8722
```

```
##           Neg Pred Value : 0.6908
```

```
##           Prevalence : 0.7607
```

```
##           Detection Rate : 0.6995
```

```
##           Detection Prevalence : 0.8020
```

```
##           Balanced Accuracy : 0.7456
```

```
##
```

```
##           'Positive' Class : <=50K
```

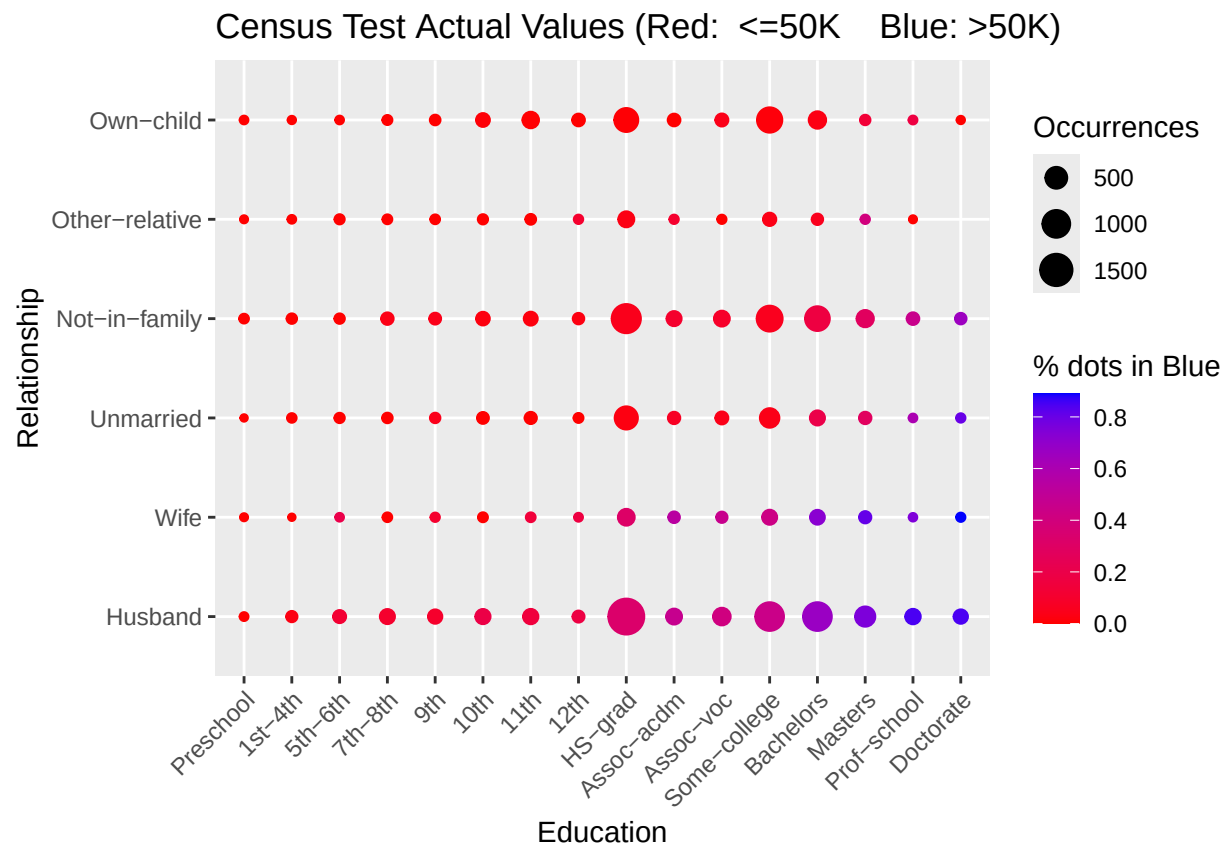
```
##
```

method	Accuracy	Parameter
1: KNN	0.8216145	K = 21
2: KNN with CV (Cross Validation)	0.8250366	K = 15
3: Decision tree	0.8211758	
4: Decision tree with CP (Complexity Parameter)	0.8271132	cp = 0.004166666666666667
5: Random forest	0.9267915	n of trees = 500, n of var = 3
6: Random forest with CV (Cross Valdidation)	0.8157063	
Random Forest (Test dataset)	0.8362681	n of trees = 500, n of var = 3

The goal of having 80+ percent accuracy has been confirmed with the test dataset.

The following graph shows the actual values of Income for the Census Test dataset. Because this is a two dimensional graph, the graph is illustrative and only two predictors out of the nine are shown, Education and Relationship.

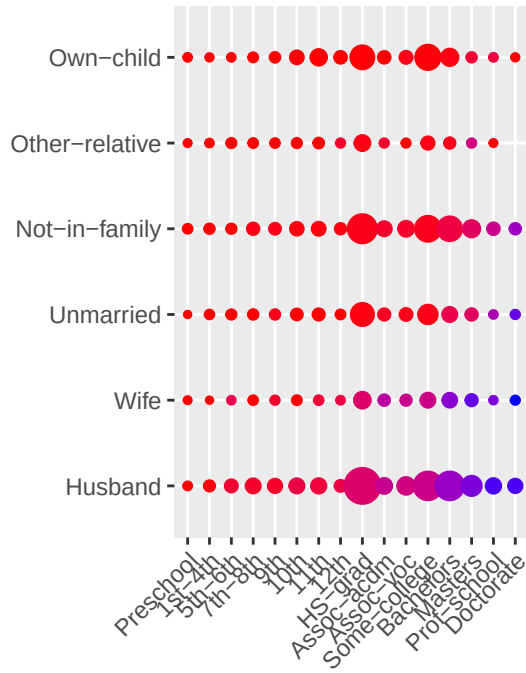
Every dot shows the frequency of occurrences that have “>50K” as income in blue as a percentage of the total number of records for that combination of Education and Relationship. The size of the dot represents the number of records for that combination of Education and Relationship.



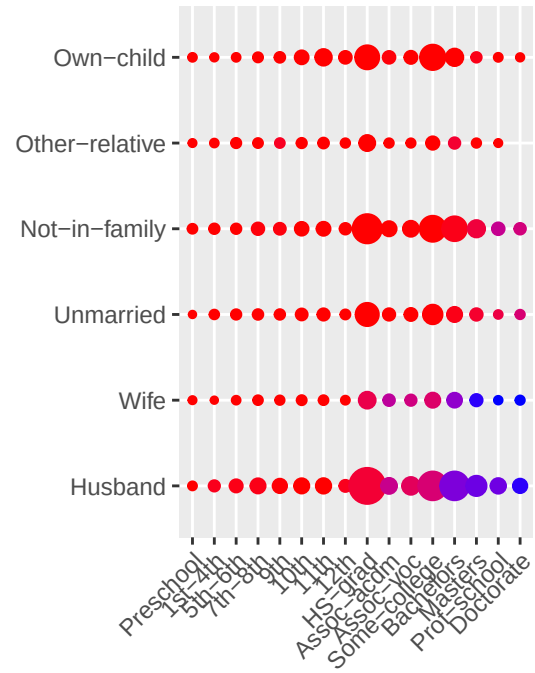
The following graph grid shows an illustrative visual comparison between the actual values of the census test dataset and a graph generated with the Test predicted values of the model with the highest Accuracy (Random Forest). This is done with just two predictors so it can be shown in a graph.

Census Test values and Random Forest predictions on Income

Census Test Values



Random Forest (Prediction)



4 CONCLUSION

In this analysis, six models were considered and accuracy was used to measure the predictive power of each model.

The models were increasing accuracy to get to the maximum with the Random Forest model.

Using Random Forest, an Income prediction model was built with an Accuracy on the test set of:

Accuracy: 92.68 %

One potential limitation of the analysis is that the dataset has relatively few records with income >50K in comparison to records with Income <=50K. This maybe a problem of Prevalence (when the ratio of Trues is close to 0 or 1).

A couple of ways to address the issue of prevalence are:

- generating a sample without prevalence
- analyzing sensitivity and specificity

An additional step that may improve the prediction is to ensemble all six models into one. Ensembles combine multiple machine learning algorithms into one model to improve predictions (2).

5 REFERENCES

1. Becker, B. & Kohavi, R. (1996). Adult [Dataset]. UCI Machine Learning Repository. <https://archive.ics.uci.edu/dataset/2/adult>
2. Irizarry, R. Introduction to Data Science - Data Analysis and Prediction Algorithms with R (32.5 Ensembles) <https://rafalab.dfci.harvard.edu/dsbook/machine-learning-in-practice.html#ensembles>